

Lab 05

1D Array

1 Review

- 1D arrays provide a way to store multiple elements of the same type in a contiguous block of memory.
 - + Arrays are fixed in size, and elements are accessed via indices starting at 0.
 - + They are ideal for tasks where a collection of related data needs to be processed, such as lists of numbers, strings, or objects...

2 Requirements

2.1 In-class assignment

2.1.1. P01

Input an array of integers with N elements, find the largest even number and the smallest odd number.

Input Data:

Includes 2 lines.

Line 1: An integer, the number of elements in the array.

Line 2: N integers, separated by a space.

Output Data:

A single line, consisting of the largest even number and the smallest odd number, separated by a space

Example:

Input	Output
5 2 6 3 8 5	8 3

2.1.2. P02

Input a sequence of N positive integers with values ranging from 1 to 100, calculate the average value of the even numbers in the sequence.

Input Data:

Includes 2 lines.

Line 1: An integer, the number of elements in the array.

Line 2: N integers, separated by a space.

Output Data:

The average value of the even numbers, rounded to 2 decimal places.

Example:

Input	Output
5 2 6 3 8 5	5.33

2.1.3. P05

Input an array of N integers, with $1 \leq N \leq 100$, sort the even numbers in the array in ascending order.

Input Data:

Includes 2 lines.

Line 1: An integer, the number of elements in the array.

Line 2: N integers, separated by a space.

Output Data:

Includes 2 lines.

Line 1: An integer, the number of elements in the array.

Line 2: The array with the even numbers sorted in ascending order.

Example:

Input	Output
5 2 6 1 4 7	5 2 4 1 6 7

2.2 Homework

2.2.1. P04

Input an array of N integers, with $1 \leq N \leq 100$, and print all the prime numbers in the array.

Input Data:

Includes 2 lines.

Line 1: An integer, the number of elements in the array.

Line 2: N integers, separated by a space.

Output Data:

Includes 2 lines.

Line 1: An integer, the number of prime numbers.

Line 2: The prime numbers, on the same line, separated by a space.

Example:

Input	Output
5 2 6 1 4 7	5 2 7

2.2.2. P08

Input a one-dimensional array of n positive integers. Remove k consecutive elements from the array starting at a given position i .

Input Data:

Line 1: Enter an integer $0 < n \leq 10^9$ (size of the array), an integer $0 \leq i \leq n-1$ (position i), and $k \geq 0$ (number of elements to remove).

Line 2: Enter n positive integers into the array.

Output Data:

Print the array after removal.

Example:

Input	Output
7 3 2 12 67 45 87 678 0 2	12 67 45 0 2
7 2 1 12 67 45 87 678 0 2	12 67 87 678 0 2
10 3 4 9 18 76 87 54 65 67 34 23 21	9 18 76 34 23 21

2.2.3. P12

Input a one-dimensional array of n positive integers. Find the second largest number in the array.

Input Data:

Line 1: Enter an integer $0 < n \leq 10^9$ (size of the array).

Line 2: Enter n positive integers into the array.

Output Data:

Print the second largest number.

Example:

Input	Output
8 5 12 5 45 87 678 5	87

2.2.4. P26

Given two arrays a and b with m and n elements respectively, where the elements in each array are distinct, find the common values that appear in both arrays. Extension: Assume there may be duplicate elements.

Input Data:

Line 1: Enter integers $0 < m, n \leq 10^9$ (the number of elements in the arrays).

Line 2: Enter array a, and array b on separate lines.

Conditions: The elements in each array are distinct from each other.

Output Data:

Find and print the values that appear in both arrays. Extension: Assume there may be duplicate elements.

Example:

Input	Output
7 6 3 5 12 67 80 45 678 5 23 12 45 56 67	5 12 67 45
7 6 3 5 12 67 67 45 678 5 23 12 45 12 67	5 12 12 67 67 45

2.2.5. P03 (in “W04-Problems-Structure and File handling.docx”)

Define a struct Course (course id, e.g. CSC10001, course name, e.g. Introduction to Programming, and an array of students, status, e.g. open, or close, maximum number of students in the course, minimum number of students in the course) and the following functions.

- P03-W04-1. Input a course (from text file).
- P03-W04-2. Output a course: id, name, students' info (to text file)
- P03-W04-3. Add a student to a course. Check the maximum number of students in the course condition.
- P03-W04-4. Remove a student to a course. Check minimum number of students in the course to update the course status: open or close.
- P03-W04-5. Get the current date in your computer. Find all students who were born in this month and print the result.
- P03-W04-6. Get the current date in your computer. Find all students who were born in this date and print the result.
- P03-W04-7. Get the current date in your computer. Find all students who are legal to have driving licences (≥ 18 years old) print the result.
- P03-W04-8. Find all students are in K19 class and print the result (to text file).
- P03-W04-9. Find a student by a given id and print the result (to text file).
- P03-W04-10. Find a student by a given name (“thanh” vs “Tuan-Thanh Ho”). Then print the result (to text file).
- P03-W04-11. Sort students by id. Then print the result (to text file).
- P03-W04-12. Sort students by first name. Then print the result (to text file).
- P03-W04-13. Sort students by gpa. Then print the result (to text file).
- P03-W04-14. Sort students by dob. Then print the result (to text file).

3 Guidelines

Create your project with appropriate .h/.cpp files.

Write at least 1 function for each problem.

Write a main() function, which a menu that allows user to choose a problem to be executed If that user chooses 0, exit the program

```
// <YOUR STUDENT ID>
// <YOUR FULL NAME>
// <YOUR CLASS>
```

```
// Test case 1
```

```
// Input:
```

```
// Output:
```

```
...
```

```
// Test case 3
```

```
// Input:
```

```
// Output:
```

```
int main(){
    //your code here

    return 0;
}
```